

Lojistik Regresyon ve Matematiksel Model

Lojistik regresyon, makine öğrenmesinde **ikili sınıflandırma** problemleri için kullanılan bir modeldir. Adında "regresyon" kelimesi geçse de, aslında bir sınıflandırma algoritmasıdır.

Amacı, bir olayın gerçekleşme **olasılığını** tahmin etmektir. Bunu, girdi değişkenleri ve bir çıktı sınıfı (0 veya 1) arasındaki ilişkiyi modelleyerek yapar.

Mesela Bir tümörün iyi huylu (0) mu yoksa kötü huylu (1) mu olduğu.

Neden Lineer Regresyon Yeterli Değil?

Lineer regresyon, **sürekli (continuous)** bir çıktıyı tahmin etmek için kullanılır. Örneğin, bir evin boyutuna ve yaşına göre fiyatını tahmin etmek gibi. Bu model, girdilerle çıktı arasında doğrusal bir ilişki kurar ve bu ilişkiye dayanarak sonsuz sayıda olası değerden birini tahmin eder.

İkili sınıflandırma ise **kesikli (discrete)** ve sadece iki olası çıktıya sahip bir problemi çözer. Yani, "evet" veya "hayır" gibi belirli ve ayrık bir karar verir.

*Lineer Regresyon sonsuz sayıda tahmin üretir. 1.5, -3, .0.7 gibi de üretebilir ama bizim istediğimiz 0 ve 1.

lineer regresyon bir sayı **tahmin ederken**, **lojistik regresyon** (ve diğer sınıflandırma algoritmaları) bir **olasılık** tahmin eder

–Mesela bir hastalık testi için hasta olanlar 1 olmayanlar 0 ile işaretlensin. Kan basıncına bakarak hastalık hakkında tahmin yapalım. Kan basıncı yüksek olanlar hasta kabul edilsin.

Eğer **lineer regresyon** kullanırsak, kan basıncı ile hastalık durumu arasında bir doğru çizeriz. Bu doğru, hastaların kan basıncı değerlerine ne kadar bağlı olduğunu gösterir

Şimdi senaryoyu biraz değiştirelim ve veri setimize birkaç **uç değer** ekleyelim:

- Normalde kan basıncı düşük olması beklenen (0 etiketli) bir hasta, yanlışlıkla çok yüksek bir kan basıncı değeri ile veri setine girmiş olsun.
- Ya da tam tersi, kan basıncı çok yüksek olması beklenen (1 etiketli) bir hasta, yanlışlıkla normal kan basıncıyla veri setine girmiş olsun.

Lineer regresyon, en küçük kareler yöntemini (least squares method) kullanarak bu doğrunun tüm noktalara olan uzaklığını minimize etmeye çalışır. Bu yüzden, veri setindeki bu **yanlış etiketlenmiş veya anormal (uç) veriler**, regresyon doğrusunu kendilerine doğru çekerler.

Bu durum, sınıflandırma açısından kritik bir hataya yol açar. Regresyon doğrusu, artık iki sınıfı (0'lar ve 1'ler) en iyi şekilde ayıramaz. **Sınır çizgisi** olması gereken yer hatalı bir şekilde konumlanır, bu da yeni veriler geldiğinde modelin yanlış tahminler yapmasına neden olur.

Nasıl Çalışır?

Lojistik regresyon, sigmoid fonksiyonunu kullanarak bu olasılıkları hesaplar. İşte adımları:

- Doğrusal Kombinasyon:** Öncelikle, doğrusal regresyonda olduğu gibi, girdilerin (özelliklerin) ağırlıklı toplamı hesaplanır. Bu toplam, bir **z** değeri verir: $z = \text{ağırlık}_1 \times \text{özellik}_1 + \text{ağırlık}_2 \times \text{özellik}_2 + \dots + \text{sabit}$
- Sigmoid Dönüşümü:** Elde edilen **z** değeri, sigmoid fonksiyonuna sokulur. Sigmoid fonksiyonu, herhangi bir değeri alıp onu 0 ile 1 arasına sıkıştırır. Bu çıktıyı bir olayın gerçekleşme olasılığı olarak yorumlayabiliriz. $\text{Olasılık} = \frac{1}{1 + e^{-z}}$
- Karar Verme:** Elde edilen olasılık değeri, bir eşik değeriyle (genellikle 0.5) karşılaştırılır.
 - Eğer olasılık 0.5'ten büyükse, tahmin **1** olur.
 - Eğer olasılık 0.5'ten küçükse, tahmin **0** olur.

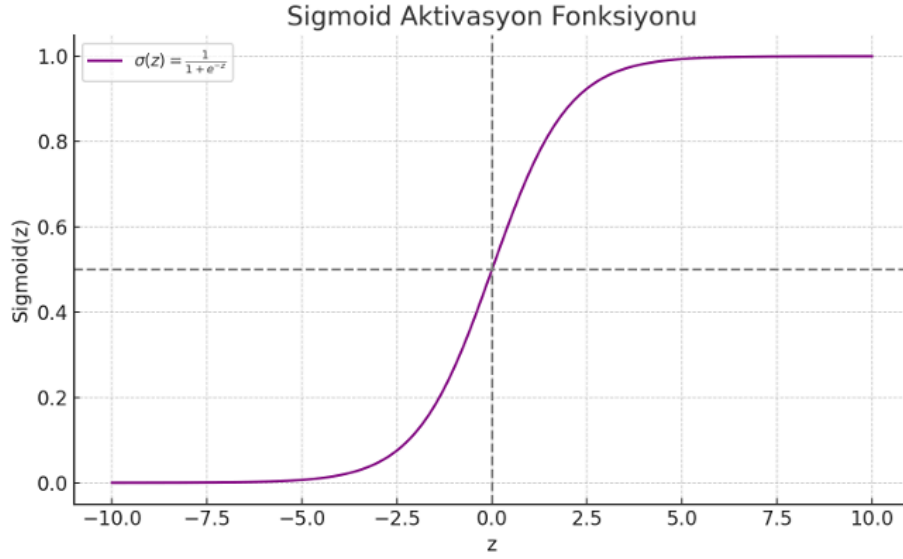
Sigmoid Fonksiyonu (Aktivasyon fonk)

Her türlü girdi değerini (pozitif, negatif, büyük, küçük) alıp, çıktısını **0 ile 1 arasında bir değere dönüştüren** özel bir matematiksel fonksiyondur.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Olasılık Çıktısı Sağlar: Sınıflandırma modelimiz, kan basıncı gibi bir dizi girdi olarak bir sayısal değer (z) hesaplar. Sigmoid fonksiyonu, bu skoru alarak onu 0 ile 1 arasına sıkıştırır. Elde edilen değer, hastanın hastalığa sahip olma **olasılığını** temsil eder.

Sınıflandırma Sınırı Oluşturur: B olasılığı bir karar eşiği (genellikle 0.5) ile karşılaştırarak kesin bir sınıflandırma kararı verebiliriz.



Hipotez Fonksiyonu

Lojistik regresyonun hipotez fonksiyonu, doğrusal regresyon sonucuna sigmoid fonksiyonu uygulanarak elde edilir:

$$h_{\theta}(x) = \sigma(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n)$$

Maliyet Fonksiyonu

Lineer regresyondaki ortalama kare hatayı (MSE) kullanmak, lojistik regresyonda konveks olmayan bir fonksiyon oluşturur. Bu nedenle, **log-loss** fonksiyonu tercih edilir:

Bu fonksiyon konvekstir, yani tek bir global minimuma sahiptir

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))]$$

Gradient Descent

Lineer regresyon ve lojistik regresyon gibi makine öğrenmesi modelleri, en iyi performansı elde etmek için en uygun parametreleri bulmaya çalışır. **Gradyan düşüş (gradient descent)**, bu parametreleri bulmak için kullanılan güçlü bir optimizasyon algoritmasıdır.

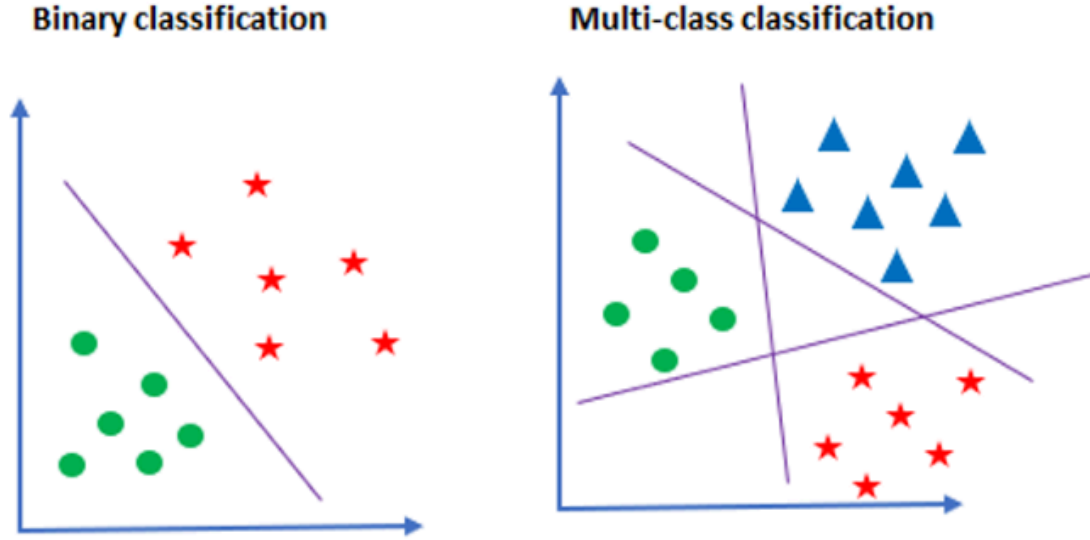
Modelin parametrelerini, **kayıp fonksiyonunun (loss function)** değerini en aza indirecek şekilde ayarlamayı hedefler. Kayıp değerini (MSE) sürekli olarak düşürerek en "doğru" parametre kombinasyonuna ulaşmaya çalışır.

****Her iterasyonda, parametreler küçük bir adım atarak MSE yi azaltır.**

****Algoritma, parametrelerin en iyi değerlerini bulduğunda, modelin tahminleri gerçek verilere en yakın hale gelir.**

Lojistik Regresyon For Multi Classification

Logistic regresyon, genellikle iki sınıfı (binary classification) ayırmak için kullanılır. Ancak çoklu sınıf problemlerinde (multi-class classification) de yaygın olarak uygulanabilir. Bu durumda **One-vs-Rest (OvR)** veya **One-vs-All** stratejisi kullanılır.



1. model için ;Modelin amacı, bu iki sınıfı ayırmak için en iyi sınır çizgisini (mor çizgi) bulmaktır. Bu çizgi, bir tarafta yeşil noktaları, diğer tarafta ise kırmızı yıldızları bırakacak şekilde konumlandırılmıştır.
Bu çizginin amacı, veriye en yakın olmak değil, sınıfları en net şekilde ayırmaktır. Linear Regresyondaki best fit line'in amacı→**noktaların kendisine en yakın olmaktır.**
Bu iki çizgi karıştırılmamalıdır.
2. Bu problemde, verileri ayırmak için birden fazla sınıra (mor çizgiler) ihtiyaç duyulur. Her bir çizgi, bir sınıfı diğerlerinden ayırmaya yardımcı olur.

Bu iki görsel arasındaki temel fark, birinin iki sınıfı tek bir çizgiyle ayırmaya çalışırken, diğerinin üç sınıfı ayırmak için birden fazla çizgi kullanmasıdır. Bu, makine öğrenmesinde problem türüne göre modelin nasıl bir yaklaşımla eğitilmesi gerektiğini çok iyi özetliyor.

One-vs-Rest (OVR) Yaklaşımı

Çok sınıflı bir problemi birden fazla bağımsız ikili sınıflandırma problemine dönüştürür. Her bir sınıf için ayrı bir ikili sınıflandırıcı eğitir.

Diyelim ki 5 sınıfımız var: Sınıf A, B, C, D ve E.

Bu durumda, her bir sınıf için ayrı bir lojistik regresyon modeli oluşturur ve eğitiriz:

1. Sınıf A için Bir Model Eğitme:

- **Pozitif Sınıf:** Sadece Sınıf A'ya ait veriler (etiket 1).
- **Negatif Sınıf:** Sınıf B, C, D ve E'ye ait tüm veriler (etiket 0).
- Bu veri setiyle bir lojistik regresyon modeli eğitilir. Modelin görevi, bir verinin "A sınıfına ait olup olmadığını" tahmin etmektir.

2. Sınıf B için Bir Model Eğitme:

- **Pozitif Sınıf:** Sadece Sınıf B'ye ait veriler (etiket 1).
- **Negatif Sınıf:** Sınıf A, C, D ve E'ye ait tüm veriler (etiket 0).
- Bu veri setiyle ikinci bir model eğitilir.

Böyle böyle her bir sınıf için yapılır.

Tahmin yapma aşamasında ise;

Yeni ve etiketlenmemiş bir veri geldiğinde, bu 5 modelin hepsi devreye girer. Her model, verinin kendi sınıfına ait olma olasılığını tahmin eder.

- Model A: Olasılık değeri 1
- Model B: Olasılık değeri 2
- Model C: Olasılık değeri 3
- Model D: Olasılık değeri 4
- Model E: Olasılık değeri 5

Sonuç olarak, en yüksek olasılık değerini veren modelin sınıfı, nihai tahmin olarak belirlenir. Örneğin, eğer en yüksek olasılığı Sınıf D modeli verdiyse, tahmininiz "Sınıf D" olur.

One-vs-One (OvO)

Her bir sınıf çifti için ayrı bir ikili sınıflandırma modeli eğitir. Bu, problem başına daha fazla sayıda model anlamına gelir.

Toplam K sınıf varsa $\rightarrow [K*(K-1)] / 2$ adet model oluşturulur

Her model sadece iki sınıf arasında ayrım yapar. Test verisi her modelden bir "oy" alır; hangi sınıf daha fazla oy alırsa o sınıf atanır..

Örnek: Eğer sınıflar A, B, ve C ise:

- Model 1: A vs B
- Model 2: A vs C
- Model 3: B vs C

Veri setindeki sınıflar arasında daha dengeli modeller oluşturulmasına olanak sağlar. Ancak model sayısı hızla artabileceği için büyük sınıf sayısına sahip veri setlerinde maliyetli olabilir.

Lojistik Regresyon – Performans Metrikleri

Confusion Matrix (Karmaşıklık Matrisi)

Confusion matrix, sınıflandırma problemlerinde modelin doğru ve yanlış tahminlerini dört kategoriye ayırır:

Bir makine öğrenmesi modelinin **sınıflandırma performansını değerlendirmek** için kullanılan bir tablodur. Özellikle ikili sınıflandırma (binary classification) problemlerinde çok faydalıdır ancak çoklu sınıflandırma için de genişletilebilir.

- **True Positive (TP)**: Gerçek değer pozitif, **tahmin de pozitif**.
- **True Negative (TN)**: Gerçek değer negatif, **tahmin de negatif**.
- **False Positive (FP)**: Gerçek değer negatif, **tahmin pozitif**.
- **False Negative (FN)**: Gerçek değer pozitif, **tahmin negatif**.

Confusion Matrix

	Actually Positive (1)	Actually Negative (0)
Predicted Positive (1)	True Positives (TPs)	False Positives (FPs)
Predicted Negative (0)	False Negatives (FNs)	True Negatives (TNs)

Accuracy (Doğruluk)

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Genel olarak modelin ne kadar doğru tahmin yaptığını gösterir. Ancak **dengesiz veri** kümelerinde yanıltıcı olabilir.

Örneğin, dolandırıcılık tespiti gibi nadir

olayları tahmin eden bir model, her zaman "dolandırıcılık değil" dese bile %99 accuracy elde edebilir, ancak bu model gerçekte hiçbir dolandırıcılığı tespit edememiştir.

Precision (Kesinlik)

Modelin **pozitif** olarak tahmin ettiği durumlardan ne kadarının gerçekten pozitif olduğunu gösterir. Diğer bir deyişle, **modelin pozitif tahminlerinin kalitesini ölçer**.

$$\text{Precision} = \frac{TP}{TP + FP}$$

**** Spam sınıflandırma:** Gerçek spam olmayan e-postaların yanlışlıkla spam olarak işaretlenmemesi gerekir. FP'nin çok çıkması sıkıntı olabilir.

–Hatta bazı durumlarda “ **FP olacağına FN olsun**” istenebilir. “*Spam değil diye düşünüp işaretlenmemesi çok bir sıkıntı olmaz, spam ise bile ben manuel olarak fark ederim onu*” diye düşünülebilir.

Ama gerçekten müşteriden gelen mesajı spam olarak işaretlerse o sıkıntı işte.

Recall (Duyarlılık)

Gerçekte pozitif olan durumların ne kadarının model tarafından doğru bir şekilde yakalandığını gösterir. Diğer bir deyişle, **modelin tüm pozitif örnekleri bulma yeteneğini ölçer**.

****Hastalık teşhisi (örneğin diyabet):** Hasta bir bireyin hasta değil olarak sınıflandırılması ciddi bir hatadır. FN azaltılmalı → Recall yüksek olmalı.

F1 Skoru

F1 Skoru, precision (kesinlik) ve recall (geri çağırma) metriklerinin dengeli bir değerlendirmesini sunan tek bir sayıdır. Özellikle sınıfların dengesiz olduğu (imbalanced classes) durumlarda, bir modelin genel performansını doğru bir şekilde değerlendirmek için sıklıkla kullanılır.

Precision ve recall'un **harmonik ortalamasıdır**. Harmonik ortalama, iki değerden daha düşük olanına daha fazla ağırlık veren bir ortalama türüdür. Bu, bir modelin hem yüksek precision'a hem de yüksek recall'a sahip olması için F1 skurunun yüksek olmasını sağlar. Tek bir metriğe odaklanarak diğerini feda etmenizi engeller.

$$F_1 = \frac{2 \cdot (\text{Precision} \cdot \text{Recall})}{\text{Precision} + \text{Recall}}$$

Örneğin, bir modelin precision'ı %100, recall'u %0 olursa, F1 skoru 0 olur. Aynı şekilde, precision %0 ve recall %100 olursa, F1 skoru yine 0 olur.

F1 Skoru Neden Önemli?

Senaryo: Nadir bir hastalığı tespit eden bir modelin performansı.

- **Model A:** 100 hastayı inceliyor. Sadece 1 kişi gerçekten hasta. Model bu kişiyi doğru teşhis ediyor, ancak sağlıklı olan 99 kişiden 5'ini yanlışlıkla "hasta" olarak etiketliyor.

- **Precision:** $TP/(TP+FP)=1/(1+5)=0.16$ (Modelin pozitif tahminleri çok güvenilir değil.)
- **Recall:** $TP/(TP+FN)=1/(1+0)=1.0$ (Gerçekten hasta olan kişiyi yakaladı.)
- **Accuracy:** $(1+94)/100=0.95$ (Genel doğruluk yanıltıcı derecede yüksek görünüyor.)
- **F1 Skoru:** $2 \times (0.16 \times 1.0) / (0.16 + 1.0) = 0.27$ (Düşük F1 skoru, modelin dengesiz performansını gösteriyor.)
- **Model B:** 100 hastayı inceliyor. Gerçekten hasta olan kişiyi yakalıyor ve sağlıklı kişilerden sadece birini yanlışlıkla hasta olarak etiketliyor.
 - **Precision:** $1/(1+1)=0.5$
 - **Recall:** $1/(1+0)=1.0$
 - **F1 Skoru:** $2 \times (0.5 \times 1.0) / (0.5 + 1.0) = 0.66$ (Daha iyi bir performans.)

Gördüğünüz gibi, Model A'nın **accuracy**'si yüksek görünse de F1 skoru, modelin düşük precision'ı nedeniyle çok düşüktür. Bu, modelin genel olarak güvenilir olmadığını gösterir. F1 skoru, modelin hem pozitif tahminlerini doğru yapma hem de gerçek pozitif vakaları kaçırmama yeteneğini bir arada değerlendirir.

Hangi Metrik Ne Zaman Önemlidir?

- **Bir e-posta spam filtresi düşünelim:**
 - **Yüksek precision** istersiniz, çünkü spam olmayan önemli bir e-postanın yanlışlıkla spam olarak işaretlenmesini (FP) istemezsiniz.
 - **Yüksek recall** da istersiniz, çünkü spam olan bir e-postanın gelen kutunuza düşmesini (FN) istemezsiniz.
- **Bir hastalık teşhis modeli düşünelim:**
 - **Yüksek recall** hayati öneme sahiptir. Hasta olan birini "hasta değil" olarak etiketlemek (FN) ciddi sonuçlar doğurabilir. Yanlış pozitifler (sağlıklı bir kişiyi hasta olarak etiketlemek) de istenmez, ancak yanlış negatifler daha tehlikeli olabilir. Bu nedenle, bu gibi durumlarda recall genellikle önceliklidir.
- **Bir suç tespiti modeli düşünelim:**
 - **Yüksek precision** önemli olabilir, çünkü suçlu olmayan birini "suçlu" olarak işaretlemek (FP) adli sonuçlar doğurabilir.